List comprehension:
List:
List comprehensions provide a concise way to create lists. Common applications are to make new lists where each element is the result of some operations applied to each member of another sequence or iterable, or to create a subsequence of those elements that satisfy a certain condition.
For example, assume we want to create a list of squares, like:

```
>>> list1=[]
>>> for x in range(10):
list1.append(x**2)
>>> list1
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

(or)
This is also equivalent to

```
>>> list1=list(map(lambda x:x**2, range(10)))
>>> list1
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

(or)
Which is more concise and redable.

```
>>> list1=[x* *2 for x in range(10)]
>>> list1
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```


Similarily some examples:

```
>>> x=[m for m in range(8)]
>>> print(x)
[0, 1, 2, 3, 4, 5, 6, 7]

>>> x=[z**2 for z in range(10) if z>4]
>>> print(x)
[25, 36, 49, 64, 81]

>>> x=[x ** 2 for x in range (1, 11) if x % 2 == 1]
>>> print(x)
[1, 9, 25, 49, 81]

>>> a=5
>>> table = [[a, b, a * b] for b in range(1, 11)]
>>> for i in table:
print(i)


[5, 1, 5]
[5, 2, 10]
[5, 3, 15]
[5, 4, 20]
[5, 5, 25]
[5, 6, 30]
[5, 7, 35]
[5, 8, 40]
[5, 9, 45]
[5, 10, 50]
```

Tuples:
A tuple is a collection which is ordered and unchangeable. In Python tuples are written with round brackets.
■ Supports all operations for sequences.
■ Immutable, but member objects may be mutable.
■ If the contents of a list shouldn't change, use a tuple to prevent items from

accidently being added, changed, or deleted.
■ Tuples are more efficient than list due to python's implementation.

We can construct tuple in many ways:
```
X=() #no item tuple
X=(1,2,3)
X=tuple(list1)
X=1,2,3,4
```

Example:
```
>>> x=(1,2,3)
>>> print(x)
(1, 2, 3)
>>> x
(1, 2, 3)
----------------------
>>> x=()
>>> x
()
---------------------------
>>> x=[4,5,66,9]
>>> y=tuple(x)
>>> y
(4, 5, 66, 9)
----------------------------
>>> x=1,2,3,4
>>> x
(1, 2, 3, 4)
```

Some of the operations of tuple are:
■ Access tuple items
■ Change tuple items
■ Loop through a tuple
■ Count()
■ Index()
■ Length()

Access tuple items: Access tuple items by referring to the index number, inside square brackets
```
>>> x=('a','b','c','g')
>>> print(x[2])
c
```
Change tuple items: Once a tuple is created, you cannot change its values. Tuples are unchangeable.

```
>>> x=(2,5,7,'4',8)
>>> x[1]=10
Traceback (most recent call last):
File "", line 1, in
x[1]=10
TypeError: 'tuple' object does not support item assignment
>>> x
(2, 5, 7, '4', 8) # the value is still the same
```

Loop through a tuple: We can loop the values of tuple using for loop

```
>>> x=4,5,6,7,2,'aa'
>>> for i in x:
print(i)


4
5
6
7
2
aa
```

Count (): Returns the number of times a specified value occurs in a tuple

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> x.count(2)
4
```

Index (): Searches the tuple for a specified value and returns the position o f where it was found

PYTHON PROGRAMMING

94

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> x.index(2)
1
```

(Or)

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> y=x.index(2)
>>> print(y)
1
```

Length (): To know the number of items or values present in a tuple, we use len().

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> y=len(x )
>>> print(y)
12
```

Tuple Assignment

Python has tuple assignment feature which enables you to assign more than one variable at a time. In here, we have assigned tuple 1 with the college information like college name, year, etc. and another tuple 2 with the values in it like number (1, 2, 3… 7).

For Example,

Here is the code,

- >>> tup1 = ('', 'eng college','2004','cse', 'it','csit');
- >>> tup2 = (1,2,3,4,5,6,7);

■ >>> print(tup1[0])
■
■ >>> print(tup2[1:4])
■ (2, 3, 4)
Tuple 1 includes list of information of
Tuple 2 includes list of numbers in it
PYTHON PROGRAMMING
95
We call the value for [0] in tuple and for tuple 2 we call the value between 1 and 4
Run the above code- It gives name for first tuple while for secon d tuple it gives
number (2, 3, 4)
Tuple as return values:
A Tuple is a comma separated sequence of items. It is created with or without (). Tuples are immutable.

```
# A Python program to return multiple values from a method using tuple

# This function returns a tuple
def fun():
str = " college "
x = 20
return str, x; # Return tuple, we could also
# write (str, x)
# Driver code to test above method
str, x = fun() # Assign returned tuple
print(str)
print(x)
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/tupretval.py
college
20

■ Functions can return tuples as return values.

```
def circleInfo(r):
""" Return (circumference, area) of a circle of radius r """
c = 2 * 3.14159 * r
a = 3.1415 9 * r * r
return (c, a)
print(circleInfo(10))
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/functupretval.py
(62.8318, 314.159)
PYTHON PROGRAMMING
96

```
def f(x):
y0 = x + 1
y1 = x * 3
y2 = y0 ** y3
return (y0, y1, y2)
```

Tuple comprehension:
Tuple Comprehensions are special: The result of a tuple comprehension is special. You might expect it to produce a tuple, but what it does is produce a special "generator" object that we can iterate over.

For example:
>>> x = (i for i in 'abc') #tuple comprehension
>>> x
at 0x033EEC30>

>>> print(x)
at 0x033EEC30>

You might expect this to print as ('a', 'b', 'c') but it prints as
at 0x02AAD710> The r esult of a tuple comprehension is not a tuple: it is actually a generator. The only thing that you need to know now about a generator now is that you can iterate over it, but ONLY ONCE.
So, given the code

>>> x = (i for i in 'abc')
>>> for i in x:
print (i)

a
b
c
Create a list of 2 -tuples like (number, square):
>>> z=[(x, x**2) for x in range(6)]
>>> z
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
PYTHON PROGRAMMING
97

Set:
Similarly to list comprehensions , set comprehensions are also supported:

>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}

>>> x={3*x for x in range(10) if x>5 }
>>> x
{24, 18, 27, 21}

Dictionaries:

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.
■ Key-value pairs
■ Unordered
We can construct or create dictionary like:
X={1:'A',2:'B',3:'c'}
X=dict([('a',3) ('b',4)]
X=dict('A'=1,'B' =2)

Example :
>>> dict1 = {"brand":"","model":"college","year":2004}
>>> dict1
{'brand': '', 'model': 'college', 'year': 2004}

Operations and methods:

Methods that are available with dictionary are tabulated below. Some of them have already been used in the above examples.
Method Description
clear() Remove all items form the dictionary.
PYTHON PROGRAMMING
98
copy() Return a shallow copy of the dictionary.
fromkeys(seq[, v]) Return a new dictionary with keys from seq and value equal to v (defaults to None).
get(key[,d]) Return the value of key. If key doesnot exit, return d (defaults to None).
items() Return a new view of the dictionary's items (key, value).
keys() Return a new view of the dictionary's keys.
pop(key[,d]) Remove the item with key and return its value or d if key is not found. If d is not provided and key is not found, raises KeyError.
popitem() Remove and return an arbitary item (key, value). Raises KeyError if the dictionary is empty.
setdefault(key[,d]) If key is in the dictionary, return its value. If not, insert key with a value of d and return d (defaults to None).
update([other]) Update the dictionary with the key/value pairs from other, overwriting existing keys.
values() Return a new view of the dictionary's values

Below are some dictionary operations:
PYTHON PROGRAMMING
99
To access specific value of a dictionary, we must pass its key,

```
>>> dict1 = {"brand":"","model":"college","year":2004}
>>> x=dict1["brand"]
>>> x
''
```

To access keys and values and items of dictionary:

```
>>> dict1 = {"brand":"","model":"college","year":2004}
>>> dict1.keys()
dict_keys(['brand', 'model', 'year'])
>>> dict1.values()
dict_values(['', 'college', 2004])
>>> dict1.items()
dict_items([('brand', ''), ('model', 'college'), ('year', 2004)])
```

```
>>> for items in dict1.values():
print(items)



college
2004

>>> for items in dict1.keys():
print(items)
```

brand
model
year

```
>>> for i in dict1.items():
print(i)
```

```
('brand', '')
('model', 'college')
('year', 2004)
```

Some more operations like:
■ Add/change
PYTHON PROGRAMMING
100
■ Remove
■ Length
■ Delete

Add/change values: You can change the value of a specific it em by referring to its key name

```
>>> dict1 = {"brand":"","model":"college","year":2004}
>>> dict1["year"]=2005
>>> dict1
{'brand': '', 'model': 'college' , 'year': 2005}
```

Remove(): It removes or pop the specific item of dictionary.

```
>>> dict1 = {"brand":"","model":"college","year":2004}
>>> print(dict1.pop("model"))
college
>>> dict1
{'brand': '', 'year': 2005}
```

Delete: Deletes a particular item.

```
>>> x = {1:1, 2:4, 3:9, 4:16, 5:25}
>>> del x[5]
>>> x
```

Length: we use len() method to get the length of dictionary.

```
>>>{1: 1, 2: 4, 3: 9, 4: 16}
{1: 1, 2: 4, 3: 9, 4: 16}
>>> y=len(x)
>>> y
4
```

Iterating over (key, value) pairs:

```
>>> x = {1:1, 2:4, 3:9, 4: 16, 5:25}
>>> for key in x:
print(key, x[key])
```

```
1 1
2 4
3 9
PYTHON PROGRAMMING
101
4 16
5 25
>>> for k,v in x.items():
print(k,v)


1 1
2 4
3 9
4 16
5 25
```

List of Dictionaries:

```
>>> customers = [{"uid":1,"name":"John"},
{"uid":2,"name":"Smith"},
{"uid":3,"name":"Andersson"},
]
>>> >>> print(customers)
[{'uid': 1, 'name': 'John'}, {'uid': 2, 'name': 'Smith'}, {'uid': 3, 'name': 'Andersson'}]

## Print the uid and name of each customer
>>> for x in customers:
print(x["uid"], x["name"])


1 John
2 Smith
3 Andersson

## Modify an entry , This will change the name of customer 2 from Smith to Charlie
>>> customers[2]["name"]="charlie"
>>> print(customers)
[{'uid': 1, 'name': 'John'}, {'uid': 2, 'name': 'Smith'}, {'uid': 3, 'name': 'charlie'}]

## Add a new field to each entry

>>> for x in customers:
x["password"]="123456" # any initial value


>>> print(customers)
PYTHON PROGRAMMING
102
[{'uid': 1, 'name': 'John', 'password': '123456'}, {'uid': 2, 'name': 'Smith', 'password':
'123456'}, {'uid': 3, 'name': 'charlie', 'password': '123456'}]

## Delete a field
>>> del customers[1]
>>> print(customers)
[{'uid': 1, 'name': 'John', 'password': '123456'}, {'uid': 3, 'name': 'charlie', 'password':
```

'123456'}]

```
>>> del customers[1]
>>> print(customers)
[{'uid': 1, 'name': 'John', 'password': '123456'}]

## Delete all fields

>>> for x in customers:
del x["uid"]


>>> x
{'name': 'John', 'password': '123456'}
```

Comprehension:

Dictionary comprehensions can be used to create dictionaries from arbitrary key and value expressions:

```
>>> z={x: x**2 for x in (2,4,6)}
>>> z
{2: 4, 4: 16, 6: 36}

>>> dict11 = {x: x*x for x in range(6)}
>>> dict11
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```


UNIT – V

FILES, EXCEPTIONS, MODULES, PACKAGES

Files and exception: text files, reading and writing files, command line arguments, errors and exceptions, handling exceptions, modules (datetime, time, OS , calendar, math module), Explore packages.

Files, Exceptions, Modules, Packages :

Files and exception :

A file is some information or data which stays in t he computer storage devices. Python gives you easy ways to manipulate these files. Generally files divide in two categories, text file and binary file. Text files are simple text where as the binary files contain binary data which is only readable by computer.

■ Text files: In this type of file, Each line of text is terminated with a special character called EOL (End of Line), which is the new line character (‘ \n’) in python by default.

■ Binary files: In this type of file, there is no terminator for a line and the data is stored after converting it into machine understandable binary language.

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception . An exception is a Python object that represents an error.

Text files:

We can create the text files by using the syntax:

Variable name =open (“file.txt”, file mode)

For ex: f= open (" hello .txt","w+")

■ We declared the variable f to open a file named hello .txt. Open takes 2 arguments, the file that we want to open and a string that represents the kinds of permission or operation we want to do on the file
■ Here we used "w" letter in our argument, which indicates write and the plus sign that means it will create a file if it does not exist in library

■ The available option beside "w" are "r" for read and "a" for append and plus sign means if it is not there then create it

File Modes in Python :
Mode Description
'r' This is the default mode. It Opens file for reading.
'w' This Mode Opens file for writing.
If file does not exist, it creates a new file.
If file exists it truncates the file.
'x' Creates a new file. If file already exists, the operation fails.
'a' Open file in append mode.
If file does not exist, it creates a new file.
't' This is the default mode. It opens in text mode.
'b' This opens in binary mode.
'+' This will open a file for reading and writing (updating)

Reading and Writing files:

The following image shows how to create and open a text file in notepad from command prompt

(or)

Hit on enter then it shows the following whether to open or not?

Click on “yes” to open else “no” to cancel

Write a python program to open and read a file

```
a=open(“one.txt”,”r”)
print(a.read())
```

```
a.close()
```

Output:

C:/Users//AppData/Local/Programs/Python/Python38 -32/filess/f1.py
welcome to python programming

(or)

Note: All the program files and text files need to saved together in a particular file then only the program performs the operations in the given file mode

f.close() ---- This will close the instance of the file somefile .txt stored

Write a python program to open and write “hello world” into a file?

```
f=open("1 .txt","a")
f.write("hello world")
f.close()
```

Output:

(or)

Note: In the above program the 1.txt file is created automatically and adds hello world into txt file

If we keep on executing the same program for more than one time then it append the data that many times

Write a python program to write the content “hi python programming” for the existing file .

```
f=open("1.txt",'w')
f.write("hi python programming")
f.close()
```

Output:

In the above program the hello txt file consist of data like

But when we try to write some data on to the same file it overwrites and saves with the current data (check output)

Write a python program to open and write the content to file and read it .

```
fo=open("ab c.txt","w+")
fo.write("Python Programming")
print(fo.read())
fo.close()
```

Output:

(or)

Note: It creates the abc.txt file automatically and writes the data into it

PYTHON PROGRAMMING
109

Command line arguments:

The command line arguments must be given whenever we want to give the input before the start of the script, while on the other hand, raw_input() is used to get the input while the python program / script is running.

The command line arguments in python can be process ed by using either ‘sys’ module, ‘argparse’ m odule and ‘getopt’ module.

‘sys’ module :

Python sys module stores the command line arguments into a list, we can access it using sys.argv. This is very useful and simple way to read command line arguments as String.
sys.argv is the list of commandline arguments passed to the Python program. argv represents all the items that come along via the command line input, it's basically an array holding the command line arguments of our program

>>> sys.modules.keys() -- this prints so many dict elements in the form of list.

```
# Python code to demonstrate the use of 'sys' module for command line arguments

import sys

# command line arguments are stored in the form
# of list in sys.argv
argumentList = sys.argv
print(argumentList)

# Print the name of file
print(sys.argv[0])

# Print the first argument after the name of file
#print(sys.argv[1])
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/cmndlinarg.py

['C:/Users//AppData/Local/Programs/Python/Python38 -32/cmndlinarg.py']
C:/Users//AppData/Local/Programs/Python/Python38 -32/cmndlinarg.py
PYTHON PROGRAMMING
110
Note: Since my list consist of only one element at ‘0’ index so it prints only that list element, if we try to access at index position ‘1’ then it shows the error like,

IndexError: list index out of range
---

import sys

print(type(sys.argv))
print('The command line arguments are:')
for i in sys.argv:
print(i)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/symod.py ==

The command line arguments are:
C:/Users//AppData/Local/Programs/Python/Python38 -32/symod.py

# write a python program to get python version.
import sys
print("System version is:")
print(sys.version)
print("Version Information is:")
print(sys.version_info)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/s1.py =
System version is:
3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)]
Version Information is:
sys.version_info(major=3, minor=8, micro=0, releaselevel ='final', serial=0)

‘argparse’ module :

Python getopt module is very similar in working as the C getopt () function for parsing command -line parameters. Python getopt module is useful in parsing command line arguments where we want user to enter some opti ons too.

>>> parser = argparse.ArgumentParser(description='Process some integers.')

#---------
PYTHON PROGRAMMING
111
import argparse

parser = argparse.ArgumentParser()
print(parser.parse_args())

‘getopt’ module :

Python argparse module is the preferred way to parse command line arguments. It provides a lot of option such as positional arguments, default value for arguments, help message, specifying data type of argument etc

It parses the command line options and parameter list. The signature of this function is ment ioned below:

getopt.getopt(args, shortopts, longopts=[ ])

■ args are the arguments to be passed.
■ shortopts is the options this script accepts.
■ Optional parameter, longopts is the list of String parameters this function accepts which should be supported. Note that the -- should not be prepended with option names.

-h --------- print help and usage message
-m --------- accept custom option value
-d --------- run the script in debug mode

```
import getopt
import sys

argv = sys.argv[0:]
try:
opts, args = getopt.getopt(argv, 'hm:d', ['help', 'my_file='])
#print(opts)
print(args)
except getopt.GetoptError:
# Print a message or do something useful
print('Something went wrong!')
sys.exit(2)
```

PYTHON PROGRAMMING
112
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/gtopt.py ==

['C:/Users//AppData/Local/Programs/Python/Python38 -32/gtopt.py']

Errors and Exceptions:
Python Errors and Built -in Exceptions: Python (interpreter) raises exceptions when it encounters errors . When writing a program, we, more often than not, will encounter errors . Error caused by not following the proper structure (syntax) of the language is called syntax error or parsing error
ZeroDivisionError :
ZeroDivisionError in Python indicates that the second argument used in a division (or modulo) operation was zero .

OverflowError :
OverflowError in Python indicates that an arithmetic operation has exceeded the limits of the current Python runtime. This is typically due to excessivel y large float values, as integer values that are too big will opt to raise memory errors instead.
ImportError :
It is raised when you try to import a module which does not exist. This may happen if you made a typing mistake in the module name or the module doesn't exist in its standard path. In the example below, a module named "non_existing_module" is being imported but it doesn't exist, hence an import error exception is raised.
IndexError :
An IndexError exception is raised when you refer a sequence which is out of range. In the example below, the list abc contains only 3 entries, but the 4th index is being accessed, which will result an IndexError exception.
TypeError :

When two unrelated type of objects are combined, TypeErrorexception is raised.In example below, an int and a string is added, which will result in TypeError exception.
PYTHON PROGRAMMING
113
IndentationError:
Unexpected indent. As mentioned in the "expected an indentedblock" section, Python not only insists on indentation, it insists on consistentindentation. You are free to choose the number of spaces of indentation to use, but you then need to stick with it.
Syntax errors :
These are the most basic type of error. They arise when the Python parser is unable to understand a line of code. Syntax errors are almost a lways fatal, i.e. there is almost never a way to successfully execute a piece of code containing syntax errors.
Run-time error :
A run-time error happens when Python understands what you are saying, but runs into trouble when following your instructions.
Key Error :
Python raises a KeyError whenever a dict() object is requested (using the
format a = adict[key]) and the key is not in the dictionary.
Value Error :
In Python, a value is the information that is stored within a certain object. To encounter a ValueError in Python means that is a problem with the content of the object you tried to assign the value to.
Python has many built -in exceptions which forces your program to output an error when something in it goes wrong. In Python, users can define such exceptions by creating a new class. This exception class has to be derived, either directly or indirectly,
from Exception class.
Different types of exceptions:
■ ArrayIndexOutOfBoundException.
■ ClassNotFoundException.
■ FileNotFoundException.
■ IOException.
■ InterruptedException.
■ NoSuchFieldException.
■ NoSuchMethodException
PYTHON PROGRAMMING
114
Handling Exceptions:
The cause of an exception is often external to the program itself. For example, an incorrect input, a malfunctioning IO device etc. Because the program abruptly terminates on encountering an exception, it may cause damage to system resources, such as files. Hence, the exceptions should be properly handled so that an abrupt termination of the program is prevented.
Python uses try and except keywords to han dle exceptions. Both keywords are followed by indented blocks.
Syntax:
try :
#statements in try block
except :
#executed when error in try block
Typically we see, most of the times
■ Syntactical errors (wrong spelling, colon (:) missing ….),
At developer level and compile level it gives errors.

■ Logical errors (2+2=4, instead if we get output as 3 i.e., wrong output …..,),
As a developer we test the application, during that time logical error may obtained.

■ Run time error (In this case, if the user doesn't know to give input, 5/6 is ok but if the user say 6 and 0 i.e.,6/0 (shows error a number cannot be divided by zero))
This is not easy compared to the above two errors because it is not done by the system, it is (mistake) done by the user.
The things we need to observe are:
1. You should be able to understand the mistakes ; the error might be done by user, DB connection or server.
2. Whenever there is an error execution should not stop.
Ex: Banking Transaction
3. The aim i s execution should not stop even though an error occurs.

For ex:

```
a=5
b=2
print(a/b)
print(" Bye")
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex1.py
2.5
Bye
```

■ The above is normal execution with no error, but if we say when b=0, it is a critical and gives error, see below

```
a=5
b=0
print(a/b)
print("bye") #this has to be printed, but abnormal termination
```

Output:

```
Traceback (most recent call last):
File "C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex2.py", line
3, in
print(a/b)
ZeroDivisionError: division by zero
```

■ To overcome this we handle exceptions using except keyword

```
a=5
b=0
```



```
try:
print(a/b)
except Exception:
print("number can not be divided by zero")
print("bye")
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex3.py
number can not be divided by zero
bye
```

■ The except block executes only when try block has an error, check it below

```
a=5
b=2
try:
print(a/b)
except Exception:
print("number can not be divided by zero")
print("bye")
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex4.py
2.5
■ For example if you want to print the message like what is an error in a program then we use “e” which is the representation or object of an exception.

```
a=5
b=0
try:
```



```
print(a/b)
except Exception as e:
print("number can not be divided by zero",e)
print("bye")
```

Output:
C:/Users//AppData/Local/Programs/Python/Pytho n38-32/pyyy/ex5.py
number can not be divided by zero division by zero
bye
(Type of error)
Let us see some more examples:
I don’t want to print bye but I want to close the file whenever it is opened.

```
a=5
b=2
try:
print("resource opened")
print(a/b)
print("resource closed")
except Exception as e:
print("number can not be divided by zero",e)
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex6.py
resource opened
2.5
resource closed

■ Note: the file is opened and closed well, but see by changing the value of b to 0,

```
a=5
b=0
try:
print("resource opened")
print(a/b)
print("resource closed")
except Exception as e:
print("number can not be divided by zero",e)
```

Output:
C:/Users//AppData/Local/Pro grams/Python/Python38 -32/pyyy/ex7.py
resource opened
number can not be divided by zero division by zero
■ Note: resource not closed
■ To overcome this, keep print(“resource closed”) in except block, see it

```
a=5
b=0
try:
print("resource opened")
print(a/b)
```

except Exception as e:
print("number can not be divided by zero",e)
print("resource closed")
Output:

C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex8.py
resource opened
number can not be divided by zero divis ion by zero
resource closed
■ The result is fine that the file is opened and closed, but again change the value of b to back (i.e., value 2 or other than zero)
a=5
b=2
try:
print("resource opened")
print(a/b)
except Exception as e:
print("num ber can not be divided by zero",e)
print("resource closed")
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex9.py
resource opened
2.5
■ But again the same problem file/resource is not closed
■ To overcome this python has a feature called finally:

This block gets executed though we get an error or not
Note: Except block executes, only when try block has an error , but finally block executes, even though you get an exception.
a=5
b=0

try:

print("resource open")
print(a/b)
k=int(input("enter a number"))
print(k)
except ZeroDivisionError as e:
print("the value can not be divided by zero",e)
finally:
print("resource closed")
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex10.py
resource open
the value can not be divided by zero division by zero
resource closed

■ change the value of b to 2 for above program , you see the output like

C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex10.py
resource open
2.5
enter a number 6

6
resource closed

■ Instead give input as some character or string for above program, check the output

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex10.py
resource open
2.5
enter a number p
resource closed
Traceback (most recent call last):
File "C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex10.py", lin e
7, in
k=int(input("enter a number"))
ValueError: invalid literal for int() with base 10: ' p'
```



```
#--------------------
a=5
b=0

try:
print("resource open")
print(a/b)
k=int(input("enter a number"))
print(k)
except ZeroDivisionError as e:
print("the value can not be divided by zero",e)
except ValueError as e:
print("invalid input")
except Exception as e:
print("something went wrong...",e)

finally:
print("resource closed")
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex11.py
resource open
the value can not be divided by zero division by zero
resource closed
```

■ Change the value of b to 2 and give the input as some character or string (other than int)

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ex12.py
resource open
2.5
enter a number p
invalid input
resource closed
```

Modules (Date, Time, os, calendar, math):

• Modules refer to a file containing Python statements and definitions.

• We use modules to break down large programs into small manageable and organized

files. Furthermore, modules provide reusability of code.
• We can define our most used functions in a module and import it , instead of copying their definitions into different programs.
• Modular programming refers to the process of breaking a large, unwieldy programming task into separate, smaller, more manageable subtasks or modules .
Advantages :
• Simplicity: Rather than focusing on the entire problem at hand, a module typically focuses on one relatively small portion of the problem. If you're working on a single module, you'll have a smaller problem domain to wrap your head around. This makes development easier and less e rror-prone.
• Maintainability: Modules are typically designed so that they enforce logical boundaries between different problem domains. If modules are written in a way that minimizes interdependency, there is decreased likelihood that modifications to a single module will have an impact on other parts of the program. This makes it more viable for a team of many programmers to work collaboratively on a large application.
• Reusability: Functionality defined in a single module can be easily reused (through an ap propriately defined interface) by other parts of the application. This eliminates the need to recreate duplicate code.
• Scoping: Modules typically define a separate namespace , which helps avoid collisions between identifiers in different areas of a program.
• Functions , modules and packages are all constructs in Python that promote code modularization.
A file containing Python code, for e.g.: example.py, is called a module and its module name would be example.

```
>>> def add(a,b):
result=a+b
return result
"""This program adds two numbers and return the result"""“
```


Here, we have defined a function add() inside a module named example. The function takes in two numbers and returns their sum.
How to import the module is:
• We can import the definitions inside a module to another module or the Interactive interpreter in Python.
• We use the import keyword to do this. To import our previously defined module example we type the following in the Python prompt.
• Using the module name we can access the function using dot (.) operation. For Eg:

```
>>> import example
>>> example.add(5,5)
10
```

• Python has a ton of standard modules available. Standard modules can be imported the same way as we import our user-defined modules.
Reloading a module:

```
def hi(a,b):
print(a+b)
hi(4,4)
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/add.py
8
>>> import add
8
```



```
>>> import add
```

>>> import add
>>>
Python provides a neat way of doing this. We can use the reload() function inside the imp module to reload a module. This is how its done.

- >>> import imp
- >>> import my_module
- This code got executed >>> import my_module >>> imp.reload(my_module) This code got executed how its done.

>>> import imp
>>> import add
>>> imp.reload(add)
8
32/pyyy \\add.py'>
The dir() built -in function
>>> import example
>>> dir(example)
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'add']
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spe c__', '__warningregistry__', 'add', 'example', 'hi', 'imp']
It shows all built -in and user -defined modules.
For ex:

>>> example.__name__
'example'
Date time module:
# Write a python program to display date, time
>>> import datetime
>>> a=datetime.datetime(2019,5,27,6,35,40)
>>> a
datetime.datetime(2019, 5, 27, 6, 35, 40)
# write a python program to display date
import datetime
a=datetime.date(2000,9,18)
print(a)
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/d1.py =
2000 -09-18
# write a python program to display time
import datetime
a=datetime.time(5,3)
print(a)
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/d1.py =
05:03:00
#write a python program to print date, time for today and now.
import datetime

a=datetime.datetime.today()
b=datetime.datetime.now()
print(a)
print(b)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/d1.py =
2019 -11-29 12:49:52.235581
2019 -11-29 12:49:52.235581
#write a python program to add some days to your present date and print the date added.
import datetime
a=datetime.date.today()
b=datetime.timedelta(days=7)
print(a+b)
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/d1.py =
2019 -12-06
#write a python program to print the no . of days to write to reach your birthday
import datetime
a=datetime.date.today()
b=datetime.date(2020,5,27)
c=b-a
print(c)
Output:
PYTHON PROGRAMMING
127
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/d1.py =
180 days, 0:00:00
#write an python program to print da te, time using date and time functions
import datetime
t=datetime.datetime.today()
print(t.date())
print(t.time())
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/d1.py =
2019 -11-29
12:53:39.226763
Time module:
#write a python program to display time.
import time
print(time.time())
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/t1.py =
1575012547.1584706
#write a python program to get structure of time stamp .
import time
print(time.localtime(time.time()))
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/t1.py =
PYTHON PROGRAMMING
128
time.struct_time(tm_year=2019, tm_mon=11, tm_mday=29, tm_hour=13, tm_min=1, tm_sec=15, tm_wday=4, tm_yday=333, tm_isdst=0)
#write a python program to make a time stamp.
import ti me
a=(1999,5,27,7,20,15,1,27,0)
print(time.mktime(a))
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/t1.py =
927769815.0

#write a python program using sleep().
import time
time.sleep(6) #prints after 6 seconds
print("Python Lab")
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/t1.py =
Python Lab (#prints after 6 seconds)
os module:
>>> import os
>>> os.name
'nt'
>>> os.getcwd()
'C:\\Users \\ \\AppData \\Local \\Programs \\Python \\Python38 -32\\pyyy'
>>> os.mkdir("temp1")

Note: temp1 dir is created
>>> os.getcwd()
'C:\\Users \\ \\AppData \\Local \\Programs \\Python \\Python38 -32\\pyyy'
>>> open("t1.py","a")
<_io.TextIOWrapper name='t1.py' mode='a' encoding='cp1252'>
>>> os.access("t1.py",os.F_OK)
True
>>> os.access("t1.py",os.W_OK)
True
>>> os.rename("t1.py","t3.py")
>>> os.access("t1.py",os.F_OK)
False
>>> os.access("t3.py",os.F_OK)
True
>>> os.rmdir('temp1')
(or)
os.rmdir('C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/ temp1')

Note: Temp1dir is removed
>>> os.remove("t3.py")
Note: We can check with the following cmd whether removed or not
>>> os.access("t3.py",os.F_OK)
False
>>> os.listdir()
['add.py', 'ali.py', 'alia.py', 'arr.py', 'arr2.py', 'arr3.py', 'arr4.py', 'arr5.py', 'arr6.py', 'br.py',
'br2.py', 'bubb.py', 'bubb2.py', 'bubb3.py', 'bubb4.py', 'bubbdesc.py', 'clo.py', 'cmndlinarg.py',
'comm.py', 'con1.py', 'cont.py', 'cont2.py', 'd1.py', 'dic.py', 'e1.py', 'example.py', 'f1.y.py',
'flowof.py', 'fr.py', 'fr2.py', 'fr3.py ', 'fu.py', 'fu1.py', 'if1.py', 'if2.py', 'ifelif.py', 'ifelse.py',
'iff.py', 'insertdesc.py', 'inserti.py', 'k1.py', 'l1.py', 'l2.py', 'link1.py', 'linklisttt.py', 'lis.py',
'listlooop.py', 'm1.py', 'merg.py', 'nesforr.py', 'nestedif.py', 'opprec.py', 'pa raarg.py',
'qucksort.py', 'qukdesc.py', 'quu.py', 'r.py', 'rec.py', 'ret.py', 'rn.py', 's1.py', 'scoglo.py',
'selecasce.py', 'selectdecs.py', 'stk.py', 'strmodl.py', 'strr.py', 'strr1.py', 'strr2.py', 'strr3.py',
'strr4.py', 'strrmodl.py', 'wh.py', 'wh1.py ', 'wh2.py', 'wh3.py', 'wh4.py', 'wh5.py',
'__pycache__']
>>> os.listdir('C:/Users//AppData/Local/Programs/Python/Python38 -32')
['argpar.py', 'br.py', 'bu.py', 'cmndlinarg.py', 'DLLs', 'Doc', 'f1.py', 'f1.txt', 'filess',
'functupretval.py', 'funturet. py', 'gtopt.py', 'include', 'Lib', 'libs', 'LICENSE.txt', 'lisparam.py',

'mysite', 'NEWS.txt', 'niru', 'python.exe', 'python3.dll', 'python38.dll', 'pythonw.exe', 'pyyy', 'Scripts', 'srp.py', 'sy.py', 'symod.py', 'tcl', 'the_weather', 'Tools', 'tupretval.p y', 'vcruntime140.dll']

Calendar module:
PYTHON PROGRAMMING
131
#write a python program to display a particular month of a year using calendar module.
import calendar
print(calendar.month(2020,1))
Output:

# write a python program to check whether the given year is leap or not.
import calendar
print(calendar.isleap(2021))
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/cl1.py
False
#write a python program to print all the months of given year.
import calendar
print(calendar.calendar(2020,1,1,1))
Output:
PYTHON PROGRAMMING
132

math module:
# write a python program which accepts the radius of a circle from user and computes the area.
import math
r=int(input("Enter radius:"))
area=math.pi*r*r
print("Area of circle is:",area)
Output:
PYTHON PROGRAMMING
133
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/m.py =
Enter radius:4
Area of circle is: 50.26548245743669
>>> import math
>>> print("The value of pi is", math.pi)
O/P: The value of pi is 3.141592653589793
Import with renaming:
• We can import a module by renaming it as follows.
• For Eg:
>>> import math as m
>>> print("The value of pi is", m.pi)
O/P: The value of pi is 3.141592653589793

• We have renamed the math module as m. This can save us typing time in some cases.
• Note that the name math is not recognized in our scope. Hence, math.pi is invalid, m.pi is the correct implementation.
Python from...import statement:
• We can import specific names from a module without importing the module as a whole. Here is an example.
>>> from math import pi

>>> print("The value of pi is", pi)
O/P: The value of pi is 3.141592653589793
• We imported only the attribute pi from the module.
• In such case we don't use the dot operator. We could have imported multiple attributes as follows.

>>> from math i mport pi, e
>>> pi
3.141592653589793
>>> e
2.718281828459045
Import all names:
• We can import all names(definitions) from a module using the following construct.
>>>from math import *
>>>print("The value of pi is", pi)
• We imported all the definitions from the math module. This makes all names except those beginnig with an underscore, visible in our scope.
Explore packages:
• We don't usually store all of our files in our computer in the same location. We use a well-organi zed hierarchy of directories for easier access.
• Similar files are kept in the same directory, for example, we may keep all the songs in the "music" directory. Analogous to this, Python has packages for directories and modules for files.
• As our application program grows larger in size with a lot of modules, we place similar modules in one package and different modules in different packages. This makes a project (program) easy to manage and conceptually clear.
• Similar, as a directory can contain sub -directories and files, a Python package can have sub -packages and modules.
• A directory must contain a file named __init__.py in order for Python to consider it as a package. This file can be left empty but we generally place the initialization code for that package in this file.
• Here is an example. Suppose we are developing a game, one possible organization of packages and modules could be as shown in the figure below.

• If a file named __init__.py is present in a package directory, it is invoked when the package or a module in the package is imported. This can be used for execution of package initialization code, such as initialization of package -level data.
• For example __init__.py
• A module in the package can access the global by importing it in turn
• We can import modules from packages using the dot (.) operator.
• For example, if want to import the start module in the above example, it is done as follows.
• import Game. Level.start
• Now if this module contains a function named select_difficulty(), we must use the full name to reference it.
• Game.Level.start.select_difficulty(2)

• If this construct seems len gthy, we can import the module without the package prefix as follows.
• from Game.Level import start
• We can now call the function simply as follows.
• start.select_difficulty(2)

• Yet another way of importing just the required function (or class or variable) form a module within a package would be as follows.
• from Game.Level.start import select_difficulty
• Now we can directly call this function.
• select_difficulty(2)
Examples:
#Write a python program to create a package (II YEAR),sub - package(CSE),modules(studen t) and create re ad and write function to module
def read():
print("Department")
def write():
print("Student")
Output:
>>> from IIYEAR.CSE import student
>>> student.read()
Department
>>> student.write()
Student
>>> from IIYEAR.CSE.student import re ad
>>> read
PYTHON PROGRAMMING
137

>>> read()
Department
>>> from IIYEAR.CSE.student import write
>>> write()
Student
# Write a program to create and impo rt module?
def add(a=4,b=6):
c=a+b
return c
Output:
C:\Users \ \AppData \Local \Programs \Python \Python38 -32\IIYEAR \modu1.py
>>> from IIYEAR import modu1
>>> modu1.add()
10
Write a program to create and rename the existing module .

def a():
print("hello world")
a()
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/IIYEAR/exam.py
hello world
>>> import exam as ex
hello world